Kontrol Motor DC
Menggunakan ESP32 dan Blynk

Dosen Pengampu: Prof. Dr. Basuki Rahmat, S.Si., M.T.

Disusun Oleh:

Nama : Sanafa Mawaddah R. Z

NPM : 24081010153

PROGRAM STUDI INFORMATIKA FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAWA TIMUR

2026

A. Tujuan Modul

1. Implementasi IoT (Internet of Things): Menghubungkan mikrokontroler ESP32 ke jaringan WiFi dan melakukan komunikasi dua arah dengan server *cloud* Blynk.
2. Kontrol Nirkabel Jarak Jauh: Merancang antarmuka pada aplikasi Blynk (tombol dan slider) untuk mengendalikan perangkat keras keras secara nirkabel.
3. Telemetri Data: Mengirimkan umpan balik data (*feedback*) dari ESP32 ke antarmuka aplikasi, seperti menampilkan nilai Duty Cycle PWM dan simulasi nilai putaran per menit (RPM).
4. Manajemen State Perangkat: Menyusun logika pemrograman yang memisahkan status operasi (ON/OFF) dengan pengaturan parameter (kecepatan).

B. Skema Rangkaian

Integrasi ESP32 dengan Motor Driver (seperti modul L298N atau L293D) menggunakan konfigurasi pin berikut:

- IN1 (Arah Putar 1): Dihubungkan ke Pin 27 ESP32.
- IN2 (Arah Putar 2): Dihubungkan ke Pin 26 ESP32.
- ENA (Kontrol Kecepatan PWM): Dihubungkan ke Pin 12 ESP32.
- Output Motor Driver (OUT1 & OUT2) disambungkan ke terminal Motor DC.
- VCC/12V Motor Driver dihubungkan ke sumber daya (baterai/adaptor) eksternal.
- Ground (GND) Motor Driver wajib dihubungkan bersama dengan Ground ESP32 dan Ground sumber daya eksternal.

C. Konsep & Logika Sistem

Sistem beroperasi berdasarkan arsitektur *Client-Server* melalui platform IoT Blynk. ESP32 bertindak sebagai *client* yang terus mendengarkan perubahan status dari server Blynk (Blynk.run()). Interaksi pengguna tidak lagi dilakukan melalui Serial Monitor, melainkan melalui *Virtual Pins* pada dashboard/aplikasi Blynk.

Saat pengguna menekan tombol atau menggeser slider di aplikasi Blynk, server mengirimkan sinyal ke ESP32. Fungsi *callback* (BLYNK_WRITE) akan otomatis terpicu untuk mengubah variabel internal (status nyala atau nilai duty cycle). Fungsi applyMotor() kemudian bertugas menerjemahkan variabel tersebut menjadi

sinyal listrik fisik ke motor, sekaligus mengirim balik pembaruan angka ke *widget* tampilan di Blynk.

D. Daftar Perintah

- V3 (Switch/Button): Berfungsi sebagai saklar utama motor (Nilai 1 = Motor ON, Nilai 0 = Motor OFF).
- V2 (Slider): Berfungsi untuk mengatur kecepatan motor (Duty Cycle) dengan rentang nilai yang dikirim dari 0 hingga 255.
- V1 (Value Display / Gauge): Menerima data dari ESP32 untuk menampilkan nilai aktual Duty Cycle (0 - 255) yang sedang berjalan pada hardware.
- V0 (Value Display / Gauge): Menerima data simulasi kecepatan RPM motor dari ESP32 (hasil pemetaan dari duty cycle menjadi nilai 0 - 13000).

E. Source Code dan Penjelasan Logika Program

```
#define BLYNK_TEMPLATE_ID ""
#define BLYNK_TEMPLATE_NAME ""
#define BLYNK_AUTH_TOKEN "" // Sesuaikan dengan yang ada pada Blynk

#define BLYNK_PRINT Serial

// --- BAGIAN INI SUDAH DISESUAIKAN ---
char ssid[] = ""; // Nama WiFi atau hotspot yang digunakan
char pass[] = ""; // Password WiFi atau hotspot yang sedang digunakan
// ------------------------------------

#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

const int freq = 30000;
const int resolution = 8;
int dutyCycle = 0;
bool isMotorRunning = false;

void applyMotor() {
```

```
  if (isMotorRunning) {
    digitalWrite(motor1Pin1, HIGH);
    digitalWrite(motor1Pin2, LOW);
    ledcWrite(enable1Pin, dutyCycle);

    Blynk.virtualWrite(V1, dutyCycle);

    int simulasiRPM = map(dutyCycle, 0, 255, 0, 13000);
    Blynk.virtualWrite(V0, simulasiRPM);

  } else {
    digitalWrite(motor1Pin1, LOW);
    digitalWrite(motor1Pin2, LOW);
    ledcWrite(enable1Pin, 0);

    Blynk.virtualWrite(V1, 0);
    Blynk.virtualWrite(V0, 0);
  }
}

BLYNK_WRITE(V3) {
  int buttonState = param.asInt();
  if (buttonState == 1) {
    isMotorRunning = true;
    if (dutyCycle == 0) dutyCycle = 150;
    Serial.println("BLYNK: Motor dinyalakan!");
  } else {
    isMotorRunning = false;
    Serial.println("BLYNK: Motor dimatikan!");
  }
  applyMotor();
}

BLYNK_WRITE(V2) {
  dutyCycle = param.asInt();
  Serial.print("BLYNK: Slider diubah ke kecepatan: ");
  Serial.println(dutyCycle);

  if (isMotorRunning) {
    applyMotor();
  }
}
```

```
void setup() {
  Serial.begin(115200);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  ledcAttach(enable1Pin, freq, resolution);

  isMotorRunning = false;
  applyMotor();

  Serial.println("\nMenghubungkan ESP32 ke WiFi dan Server
Blynk...");
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  Serial.println("Berhasil Terhubung ke Blynk Cloud!");
}

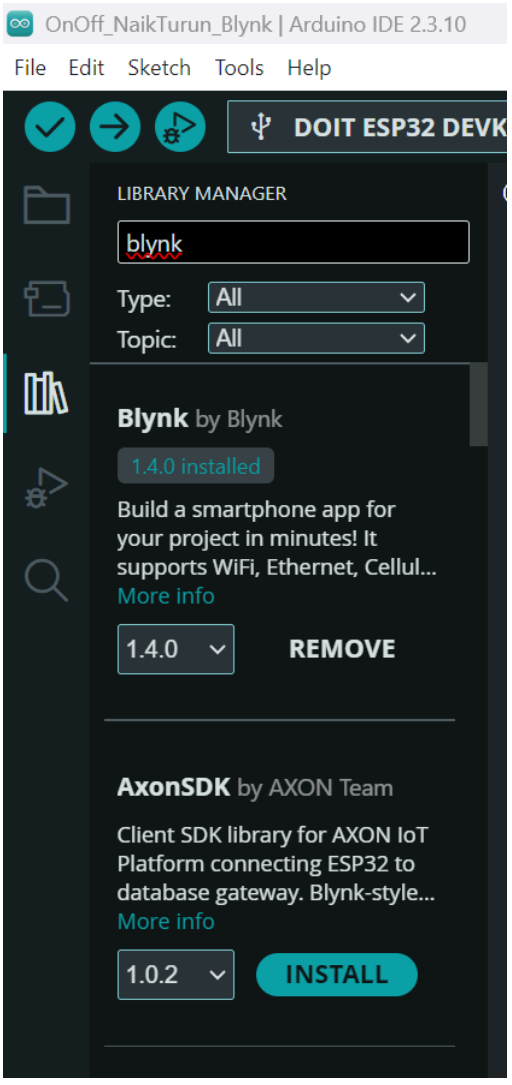
void loop() {
  Blynk.run();
}
```

Penjelasan program

1. Konfigurasi Awal & Koneksi: Program memuat pustaka WiFi dan Blynk, serta menyimpan kredensial (Auth Token, SSID, Password). Pada setup(), program menginisiasi pin, memasang antarmuka PWM (ledcAttach), dan mengeksekusi Blynk.begin() untuk memblokir sementara eksekusi sampai ESP32 berhasil terkoneksi ke jaringan dan server Blynk.
2. Loop Utama: Fungsi loop() dibuat sebersih mungkin, hanya berisi Blynk.run(). Fungsi ini secara terus-menerus memelihara koneksi dengan server dan mendengarkan instruksi masuk dari perangkat *smartphone* atau *web browser*.
3. BLYNK_WRITE(V3) (Kontrol Daya): Saat tombol V3 di aplikasi ditekan, fungsi ini dieksekusi. Jika dihidupkan (nilai 1), variabel boolean isMotorRunning bernilai *true*. Terdapat logika pintar: jika dutyCycle masih bernilai 0, sistem langsung menyesuaikannya ke angka 150 agar motor memiliki daya yang cukup untuk memulai putaran (mengatasi *stalling* pada tegangan rendah).

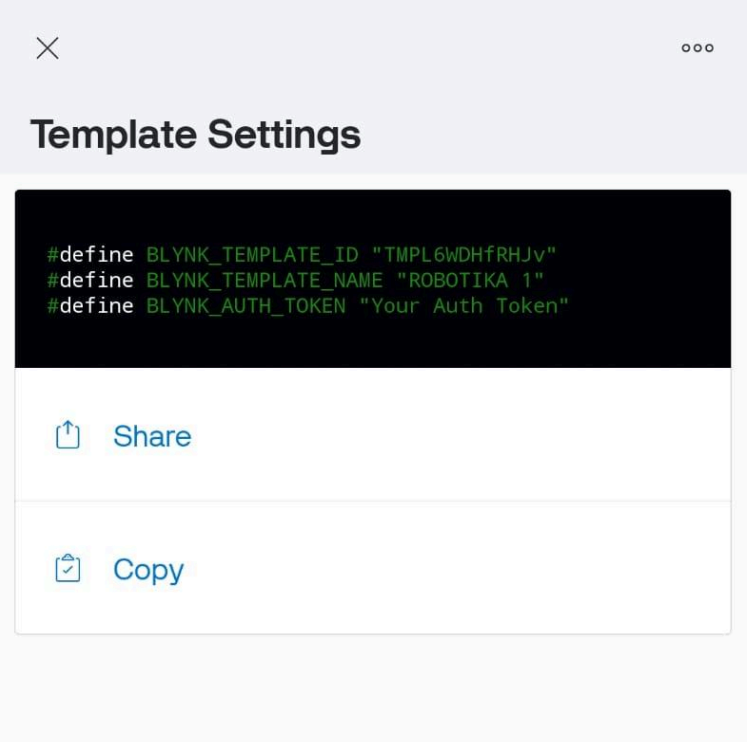
4. BLYNK_WRITE(V2) (Kontrol Kecepatan): Merespon pergeseran slider di Blynk. Nilai parameter disimpan dalam dutyCycle. Kecepatan hanya akan disinkronisasi ke *hardware* jika status isMotorRunning adalah *true*.
5. applyMotor() (Manajemen Hardware & Feedback): Mengeksekusi kendali langsung ke pin ESP32 berdasarkan state isMotorRunning. Jika motor sedang berjalan, fungsi ini mengirim nilai PWM melalui ledcWrite. Selain mengatur *hardware*, fungsi ini menggunakan perintah Blynk.virtualWrite() untuk melaporkan ulang nilai aktual ke aplikasi Blynk, beserta simulasi nilai RPM dengan menggunakan fungsi matematis map().

F. Pembuatan Device dan Pengujian Sistem

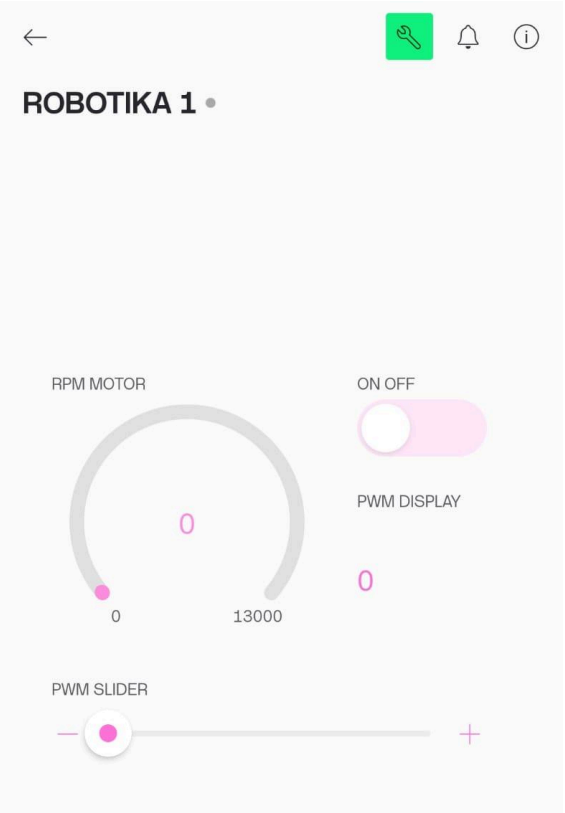


Sebelum memulai pemrograman, langkah pertama yang harus dilakukan adalah menginstal pustaka (library) agar Arduino IDE dapat mengenali perintah-perintah komunikasi IoT. Buka menu Tools > Manage Libraries, kemudian ketik "blynk" pada kolom pencarian. Pilih library Blynk

by Blynk (versi 1.4.0 atau yang terbaru) dan lakukan instalasi. Pustaka ini berisi paket *client* yang memungkinkan ESP32 terhubung ke jaringan WiFi dan server *cloud* Blynk.



Setelah membuat *Template* dan *Device* baru di *platform* web Blynk, sistem akan menghasilkan kode kredensial unik. Seperti yang terlihat pada gambar, terdapat tiga baris kode penting yaitu BLYNK_TEMPLATE_ID, BLYNK_TEMPLATE_NAME (ROBOTIKA 1), dan BLYNK_AUTH_TOKEN. Ketiga baris kode otentikasi ini wajib disalin dan di-paste ke baris paling atas pada *source code* Arduino IDE agar perangkat keras dapat dikenali dan diberikan akses masuk oleh server Blynk.



Langkah selanjutnya adalah merancang antarmuka pengendali jarak jauh pada aplikasi Blynk di *smartphone*. Antarmuka (ROBOTIKA 1) ini disusun menggunakan beberapa *widget* yang dihubungkan ke *Virtual Pin* (V-Pin):

- ON OFF (Switch): Dihubungkan ke pin V3 sebagai saklar daya utama.
- PWM SLIDER: Dihubungkan ke pin V2 untuk mengatur kecepatan secara manual.
- PWM DISPLAY (Gauge/Value): Dihubungkan ke pin V1 untuk menampilkan nilai kecepatan aktual (0-255).
- RPM MOTOR (Gauge): Dihubungkan ke pin V0 untuk menampilkan simulasi kecepatan dalam format RPM (0-13000).

```
int motor1Pin2 = 26;
```

Output

```
Writing at 0x000ebc13 [==========================> ]  98.4% 589824/599570 bytes...

Writing at 0x000ef420 [==============================] 100.0% 599570/599570 bytes...
Wrote 914464 bytes (599570 compressed) at 0x00010000 in 54.2 seconds (135.0 kbit/s).
Verifying written data...
Hash of data verified.

Hard resetting via RTS pin...
```

Setelah kode program dan pengaturan WiFi disesuaikan, dilakukan proses *upload* ke ESP32. Jendela *output console* menunjukkan proses penulisan memori (*Writing at 0x...*) berjalan lancar hingga mencapai 100%. Pesan Hard resetting via RTS pin... menandakan bahwa *firmware* berhasil ditanamkan dengan sukses tanpa ada *error* pada sintaks, dan mikrokontroler langsung melakukan *restart* untuk menjalankan program baru tersebut.

Output Serial Monitor ×

Message (Enter to send message to 'DOIT ESP32 DEVKIT V1' on 'COM5')

```
load:0x40078000,len:15592
load:0x40080400,len:3164
entry 0x4008059c

Menghubungkan ESP32 ke WiFi dan Server Blynk...
[23] Connecting to zz
[1611] Connected to WiFi
[1611] IP: 172.21.19.155
[1611]
    ___  __          __
   / _ )/ /_ _____  / /__
  / _  / / // / _ \/  '_/
 /____/_/\_, /_//_/_/\_\
        /___/ v1.4.0 on ESP32

 #StandWithUkraine    https://bit.ly/swua


[1622] Connecting to blynk.cloud:80
[1890] Ready (ping: 55ms).
Berhasil Terhubung ke Blynk Cloud!
```

```
Berhasil Terhubung ke Blynk Cloud!
BLYNK: Motor dinyalakan!
BLYNK: Slider diubah ke kecepatan: 137
BLYNK: Slider diubah ke kecepatan: 211
BLYNK: Slider diubah ke kecepatan: 255
BLYNK: Slider diubah ke kecepatan: 187
BLYNK: Slider diubah ke kecepatan: 236
BLYNK: Slider diubah ke kecepatan: 210
BLYNK: Slider diubah ke kecepatan: 185
BLYNK: Slider diubah ke kecepatan: 222
BLYNK: Motor dimatikan!
```

Pengujian terakhir dilakukan dengan memantau Serial Monitor pada *baud rate* 115200. Berdasarkan log aktivitas, ESP32 berhasil terhubung ke jaringan WiFi ("zz") dan mendapatkan alamat IP (172.21.19.155). Selanjutnya, perangkat sukses melakukan *ping* ke server blynk.cloud (55ms) dan berstatus *Ready*. Saat *widget* di aplikasi ditekan atau digeser, Serial Monitor langsung menampilkan respon *real-time* seperti "BLYNK: Motor dinyalakan!" dan perubahan nilai *slider* kecepatan secara berurutan, membuktikan bahwa komunikasi IoT dua arah telah berjalan sempurna.

G. Halis Pengujian & Analisis

1. Responsivitas Berbasis Jaringan: Berbeda dengan komunikasi Serial yang instan, latensi (jeda) eksekusi instruksi dari Blynk sepenuhnya bergantung pada kualitas dan *ping* jaringan WiFi yang digunakan oleh ESP32 serta *smartphone* pengontrol.
2. Sinkronisasi Dua Arah (Two-Way Feedback): Dengan adanya perintah Blynk.virtualWrite pada fungsi applyMotor(), aplikasi Blynk tidak hanya bertindak sebagai *remote control* searah, namun berubah menjadi *dashboard monitoring* aktif yang memvalidasi apakah perintah slider dan tombol benar-benar telah dieksekusi oleh hardware.
3. Logika Simulasi RPM: Pada pengujian, RPM aktual motor belum dibaca karena tidak terdapat sensor fisik (seperti rotary encoder). Angka RPM yang tampil di aplikasi (V0) murni merupakan estimasi matematis menggunakan fungsi map(), di mana sinyal listrik terkecil (0) dianggap 0 RPM dan maksimal (255) dianggap 13.000 RPM.
4. Stabilitas *Looping*: Absennya *delay* atau proses berat lainnya di dalam struktur void loop() mencegah terjadinya interupsi pada protokol TCP/IP Blynk.run(), sehingga ESP32 tidak mudah mengalami *disconnect* (Heartbeat Timeout) dari server *cloud*.